

JAVA LEETCODE INTERVIEW FLASHCARDS

Systematic Pattern Mastery and Code Comprehension Study Deck

This deck of 25 comprehensive cards covers the core algorithmic patterns, distributed system design components, and behavioral loops required to clear Google, Meta, and other tier-1 technical interviews. Rather than relying on rote memorization, study these **transferable mental models** to quickly categorize and solve any novel problem variant you encounter during live onsite.

CARD 01: Two Sum II (Opposite Direction Pointers)

LINEAR TOPOLOGIE

TRIGGER CONDITION

Handled a sorted array or string, and need to find a pair of elements that meet a target condition (e.g., sum, product, binary target).

MENTAL MODEL

Imagine two friends walking from opposite ends of a bench toward the middle. If their combined weight is too heavy, the heavier person (right) walks inward to decrease the sum; if too light, the lighter person (left) moves inward.

HOW TO IMPLEMENT

1. Initialize left = 0 and right = nums.length - 1.
2. Loop while left < right.
3. Calculate sum. If sum == target, return indices.
4. If sum < target, increment left (to increase next sum).
5. If sum > target, decrement right (to decrease next sum).

JAVA PRODUCTION TEMPLATE

```
public int[] solveTwoPointers(int[] nums, int target) {
    int left = 0, right = nums.length - 1;
    while (left < right) {
        int currentSum = nums[left] + nums[right];
        if (currentSum == target) {
            return new int[]{left + 1, right + 1}; // 1-based index
        } else if (currentSum < target) {
            left++;
        } else {
            right--;
        }
    }
    return new int[]{-1, -1};
}
```

TRIGGER CONDITION

Need to detect loops or cycles in a Linked List, find the start of a loop, or locate the exact middle node of a list.

MENTAL MODEL

Imagine two runners on a circular track. Runner A (Fast) runs at twice the speed of Runner B (Slow). Even if they start at different times, the fast runner will eventually overlap and pass the slow runner from behind.

HOW TO IMPLEMENT

1. Place both slow and fast pointers at the head of the list.
2. In a loop, advance slow by 1 node and fast by 2 nodes.
3. If slow == fast at any point, a cycle exists.
4. To find the middle of the list: when fast reaches the end, slow is at the exact middle.

JAVA PRODUCTION TEMPLATE

```
public boolean hasCycle(ListNode head) {  
    if (head == null) return false;  
    ListNode slow = head, fast = head;  
    while (fast != null && fast.next != null) {  
        slow = slow.next;  
        fast = fast.next.next;  
        if (slow == fast) return true; // Cycle detected  
    }  
    return false;  
}
```

TRIGGER CONDITION

Asked to compute some statistic (max, sum, distinct counts) of a contiguous subarray of a fixed size K.

MENTAL MODEL

Imagine a camera lens with a fixed focal length sliding across a horizontal panoramic landscape. As the frame moves one step to the right, you only need to process the new pixel entering on the right, and subtract the old pixel exiting on the left.

HOW TO IMPLEMENT

1. Loop through the array, accumulating values for the first K elements to establish the initial window.
2. Continue the loop from index K to nums.length - 1.
3. Add the incoming element nums[i] and remove the outgoing element nums[i - K].
4. Update your running max/global statistic at each slide step.

JAVA PRODUCTION TEMPLATE

```
public int maxSubarraySumOfSizeK(int[] nums, int k) {
    int windowSum = 0, maxSum = 0;
    for (int i = 0; i < nums.length; i++) {
        windowSum += nums[i];
        if (i >= k - 1) {
            maxSum = Math.max(maxSum, windowSum);
            windowSum -= nums[i - (k - 1)]; // Remove exiting element
        }
    }
    return maxSum;
}
```

TRIGGER CONDITION

Need to find the longest or shortest contiguous subarray that satisfies a specific dynamic constraint (e.g., $\text{sum} \geq \text{target}$, at most K distinct characters).

MENTAL MODEL

Think of an accordion. You stretch the right side out (expand window) to take in more data. Once your constraint is violated, you compress the left side in (shrink window) until the conditions are valid again.

HOW TO IMPLEMENT

1. Initialize $\text{left} = 0$, and a global variable for length/result.
2. Expand the window by iterating right from 0 to $\text{nums.length} - 1$.
3. Add $\text{nums}[\text{right}]$ to your active state (hashmap, frequency table, or sum).
4. While the constraint is violated, shrink from the left by updating state and incrementing left .
5. Update your global answer (max or min length) at each valid window state.

JAVA PRODUCTION TEMPLATE

```
public int shortestSubarray(int[] nums, int target) {
    int left = 0, minLength = Integer.MAX_VALUE, currentSum = 0;
    for (int right = 0; right < nums.length; right++) {
        currentSum += nums[right];
        while (currentSum >= target) {
            minLength = Math.min(minLength, right - left + 1);
            currentSum -= nums[left];
            left++;
        }
    }
    return minLength == Integer.MAX_VALUE ? 0 : minLength;
}
```

TRIGGER CONDITION

Asked to search for the "Next Greater Element," "Next Smaller Element," or temperature spans in an array under $O(N)$ time complexity constraints.

MENTAL MODEL

Imagine line-of-sight view from a mountain peak. If you are standing in a line of people of varying heights, your view of the horizon is blocked only by the next person who is strictly taller than you. Anyone shorter is hidden in your "shadow".

HOW TO IMPLEMENT

1. Initialize an empty Stack to hold array indices.
2. Iterate through the array.
3. While the stack is not empty and the current element `nums[i]` is greater than the element at the index stored at the top of the stack: pop the index, update its next-greater-element in the result array to `nums[i]`.
4. Push the current index `i` onto the stack.

JAVA PRODUCTION TEMPLATE

```
public int[] nextGreaterElement(int[] nums) {
    int[] result = new int[nums.length];
    java.util.Stack<Integer> stack = new java.util.Stack<>();
    java.util.Arrays.fill(result, -1);
    for (int i = 0; i < nums.length; i++) {
        while (!stack.isEmpty() && nums[i] > nums[stack.peek()]) {
            result[stack.pop()] = nums[i];
        }
        stack.push(i);
    }
    return result;
}
```

TRIGGER CONDITION

Handled list of ranges or intervals, and asked to merge overlapping boundaries, insert a new range, or calculate free/busy calendar times.

MENTAL MODEL

Think of a timeline of events. If Event A ends at 5:00 PM and Event B starts at 4:30 PM, their times overlap. You collapse them into a single continuous meeting block from A's start time to the maximum of A's or B's end times.

HOW TO IMPLEMENT

1. Sort the intervals based on their start times first.
2. Create a list to hold merged intervals.
3. Iterate through intervals. For each interval:
4. - If it doesn't overlap with the last merged interval, append it directly.
5. - If it overlaps, merge them by updating the end of the last interval to $\text{Math.max}(\text{last.end}, \text{current.end})$.

JAVA PRODUCTION TEMPLATE

```
public int[][] mergeIntervals(int[][] intervals) {
    if (intervals.length <= 1) return intervals;
    java.util.Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));
    java.util.List<int[]> merged = new java.util.ArrayList<>();
    int[] current = intervals[0];
    merged.add(current);
    for (int[] interval : intervals) {
        if (interval[0] <= current[1]) {
            current[1] = Math.max(current[1], interval[1]); // Expand end
        } else {
            current = interval;
            merged.add(current);
        }
    }
    return merged.toArray(new int[merged.size()][]);
}
```

TRIGGER CONDITION

Asked to find the K largest, K smallest, or K most frequent elements from an unsorted stream of data.

MENTAL MODEL

Think of an VIP guest list at a club door capped at size K. When a new guest arrives, if they have higher status (larger value) than the lowest-status person on the VIP list, the lowest is kicked out, keeping only the elite K inside.

HOW TO IMPLEMENT

1. Initialize a Min Heap (for K largest elements) of size K.
2. Add the first K elements to the heap.
3. For subsequent elements: if the element is strictly greater than the top of the Min Heap (the smallest VIP), pop the top and insert the new element.
4. This reduces time complexity from $O(N \log N)$ to $O(N \log K)$.

JAVA PRODUCTION TEMPLATE

```
public int[] topKElements(int[] nums, int k) {
    java.util.PriorityQueue<Integer> minHeap = new java.util.PriorityQueue<>();
    for (int num : nums) {
        minHeap.add(num);
        if (minHeap.size() > k) {
            minHeap.poll(); // Evict smallest element
        }
    }
    int[] result = new int[k];
    for (int i = 0; i < k; i++) result[i] = minHeap.poll();
    return result;
}
```

TRIGGER CONDITION

You are given K sorted arrays, lists, or database streams, and need to merge them into a single, contiguous sorted output.

MENTAL MODEL

Think of K queues at supermarket cash registers. To check out customers in chronological order, you look only at the person standing at the absolute front of each of the K queues. The customer with the earliest arrival checks out, and the next person behind them steps forward.

HOW TO IMPLEMENT

1. Initialize a Min Heap containing the first element of each of the K lists.
2. Pop the smallest node/element from the heap, and write it to your result list.
3. If the popped element had a successor node in its original list, insert that successor into the heap.
4. Repeat until the Heap is completely empty.

JAVA PRODUCTION TEMPLATE

```
public ListNode mergeKLists(ListNode[] lists) {
    java.util.PriorityQueue<ListNode> minHeap = new java.util.PriorityQueue<>((a, b) -> a.val - b.val);
    for (ListNode node : lists) {
        if (node != null) minHeap.add(node);
    }
    ListNode dummy = new ListNode(0), tail = dummy;
    while (!minHeap.isEmpty()) {
        ListNode curr = minHeap.poll();
        tail.next = curr;
        tail = tail.next;
        if (curr.next != null) minHeap.add(curr.next);
    }
    return dummy.next;
}
```

TRIGGER CONDITION

Given an array containing numbers in a strictly bounded range (e.g., 1 to N or 0 to N), and asked to locate missing, duplicated, or corrupted elements in $O(N)$ time and $O(1)$ space.

MENTAL MODEL

Imagine a classroom of N students assigned seats numbered 1 to N. Instead of sorting them alphabetically, you walk down the rows. If you find a student sitting in the wrong seat, you tap them on the shoulder and swap them with whoever is currently sitting in their correct seat. You repeat this swap chain until the row is perfectly correct.

HOW TO IMPLEMENT

1. Initialize $i = 0$.
2. At index i , check if the element $\text{nums}[i]$ is in its correct index slot (i.e., $\text{nums}[i] == \text{nums}[\text{nums}[i] - 1]$ for 1-based indexing).
3. If yes, increment $i++$.
4. If no, swap $\text{nums}[i]$ with the element at its correct target index.

JAVA PRODUCTION TEMPLATE

```
public void cyclicSort(int[] nums) {
    int i = 0;
    while (i < nums.length) {
        int correctIndex = nums[i] - 1; // 1-based conversion
        if (nums[i] > 0 && nums[i] <= nums.length && nums[i] != nums[correctIndex]) {
            int temp = nums[i];
            nums[i] = nums[correctIndex];
            nums[correctIndex] = temp;
        } else {
            i++;
        }
    }
}
```

TRIGGER CONDITION

Asked to traverse a binary tree or graph layer by layer, level by level, or find the shortest path in an unweighted grid.

MENTAL MODEL

Think of ripples spreading across a pond when a stone is thrown in. The water moves outward in perfect, concentric rings, exploring every coordinate at distance 1, then distance 2, before reaching distance 3.

HOW TO IMPLEMENT

1. Initialize a Queue and add the root node.
2. Loop while the Queue is not empty.
3. Capture the current size of the Queue (`levelSize = queue.size()`). This step is critical; it seals the boundaries of the current layer.
4. Dequeue and process exactly `levelSize` elements, pushing their left/right children into the queue.

JAVA PRODUCTION TEMPLATE

```
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new java.util.ArrayList<>();
    if (root == null) return result;
    java.util.Queue<TreeNode> queue = new java.util.LinkedList<>();
    queue.add(root);
    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> currentLevel = new java.util.ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode curr = queue.poll();
            currentLevel.add(curr.val);
            if (curr.left != null) queue.add(curr.left);
            if (curr.right != null) queue.add(curr.right);
        }
        result.add(currentLevel);
    }
    return result;
}
```

CARD 11: Tree Depth-First Search (DFS)

NON-LINEAR DFS

TRIGGER CONDITION

Asked to analyze leaf-to-root paths, find tree heights/depths, check tree symmetry, or find structural properties recursively.

MENTAL MODEL

Think of a maze crawler. Instead of looking at every path near the entrance, you walk completely down one hallway until you hit a dead-end wall, backtrack one step, and try the next deepest split.

HOW TO IMPLEMENT

1. Define base cases (e.g., if node is null, return default).
2. Recurse deep into the left subtree.
3. Recurse deep into the right subtree.
4. Combine child answers at the root and bubble the result up the recursion stack.

JAVA PRODUCTION TEMPLATE

```
public int maxDepth(TreeNode root) {  
    if (root == null) return 0; // Base Case  
    int leftDepth = maxDepth(root.left);  
    int rightDepth = maxDepth(root.right);  
    return Math.max(leftDepth, rightDepth) + 1; // Backtracking bubble up  
}
```

TRIGGER CONDITION

Given a set of tasks or classes with strict pre-requisites (Directed Acyclic Graphs), and asked to find a valid ordering to execute all items.

MENTAL MODEL

Think of a college course catalog. You cannot take Advanced Databases until you complete Database Systems, and you cannot take that until you finish Intro to CS. You build a pipeline where each prerequisite is scheduled strictly first.

HOW TO IMPLEMENT

1. Build an adjacency list of the graph and calculate the in-degree of every node (number of incoming dependency arrows).
2. Add all nodes with in-degree == 0 (no pre-requisites) to a Queue.
3. While the queue is not empty, pop a node, write it to your result list, and decrement the in-degree of all its connected neighbors.
4. If any neighbor's in-degree drops to 0, add it to the queue.

JAVA PRODUCTION TEMPLATE

```
public int[] findOrder(int numCourses, int[][] prerequisites) {
    java.util.List<List<Integer>> adj = new java.util.ArrayList<>();
    int[] inDegree = new int[numCourses];
    for (int i = 0; i < numCourses; i++) adj.add(new java.util.ArrayList<>());
    for (int[] pre : prerequisites) {
        adj.get(pre[1]).add(pre[0]);
        inDegree[pre[0]]++;
    }
    java.util.Queue<Integer> queue = new java.util.LinkedList<>();
    for (int i = 0; i < numCourses; i++) if (inDegree[i] == 0) queue.add(i);
    int[] result = new int[numCourses];
    int index = 0;
    while (!queue.isEmpty()) {
        int curr = queue.poll();
        result[index++] = curr;
        for (int neighbor : adj.get(curr)) {
            inDegree[neighbor]--;
            if (inDegree[neighbor] == 0) queue.add(neighbor);
        }
    }
    return index == numCourses ? result : new int[0];
}
```

TRIGGER CONDITION

Asked to track connected components in a network, merge social groups dynamically, or check for cycles in an undirected graph.

MENTAL MODEL

Imagine a group of scattered tribes. Every time two tribes meet, they merge. To make this efficient, each tribe nominates one "Chief." When you want to check if two people belong to the same tribe, you only check if they answer to the exact same Chief.

HOW TO IMPLEMENT

1. Maintain a parent array where `parent[i]` is initialized to `i`.
2. Implement `find(i)`: recursively traverse up to find the root/Chief. Use Path Compression to point nodes directly to the root for future $O(1)$ lookups.
3. Implement `union(i, j)`: find the roots of both. If different, set the parent of one root to the other.

JAVA PRODUCTION TEMPLATE

```
class UnionFind {
    private int[] parent;
    public UnionFind(int size) {
        parent = new int[size];
        for (int i = 0; i < size; i++) parent[i] = i;
    }
    public int find(int i) {
        if (parent[i] == i) return i;
        return parent[i] = find(parent[i]); // Path Compression
    }
    public boolean union(int i, int j) {
        int rootI = find(i), rootJ = find(j);
        if (rootI != rootJ) {
            parent[rootI] = rootJ; // Merge groups
            return true;
        }
        return false; // Loop/cycle detected
    }
}
```

TRIGGER CONDITION

Asked to generate all possible valid subsets, combinations, permutations, or configurations (e.g., N-Queens, Sudoku solver).

MENTAL MODEL

Think of a combo lock. You try the first number, then the second, then the third. If a partial combination is invalid, you immediately stop, wipe that last step, and try the next number on the previous dial.

HOW TO IMPLEMENT

1. Define a recursive helper with an active state path and a starting point.
2. Base Case: If path meets target length/condition, copy and add to global results.
3. Loop through choices: if valid, make choice (add to path).
4. Recurse to next step.
5. Undo choice (remove from path) to restore state for parallel branches.

JAVA PRODUCTION TEMPLATE

```
public void backtrack(List<List<Integer>> result, List<Integer> path, int[] nums, int start) {
    result.add(new java.util.ArrayList<>(path)); // Save current subset
    for (int i = start; i < nums.length; i++) {
        path.add(nums[i]); // Make choice
        backtrack(result, path, nums, i + 1); // Recurse
        path.remove(path.size() - 1); // Undo choice
    }
}
```

TRIGGER CONDITION

Given a set of items with weights and values, and a maximum capacity, asked to find the maximum value you can pack without exceeding weight.

MENTAL MODEL

Imagine you are packing a backpack for a hike. For each item, you must make a binary decision: pack it or leave it. You write down the optimal decisions in a logbook so that if you have the same remaining weight later, you don't calculate it from scratch.

HOW TO IMPLEMENT

1. Create a 2D grid $dp[i][w]$ where rows represent items and columns represent target capacities.
2. Base Case: Initialize first row/column to 0.
3. For each item i and weight w :
4. - If weight of item $i > w$, inherit previous: $dp[i][w] = dp[i-1][w]$.
5. - If item fits, choose max of leaving it ($dp[i-1][w]$) vs taking it ($value[i] + dp[i-1][w - weight[i]]$).

JAVA PRODUCTION TEMPLATE

```
public int knapsack(int[] weights, int[] values, int capacity) {
    int n = weights.length;
    int[][] dp = new int[n + 1][capacity + 1];
    for (int i = 1; i <= n; i++) {
        for (int w = 1; w <= capacity; w++) {
            if (weights[i - 1] <= w) {
                dp[i][w] = Math.max(dp[i - 1][w], values[i - 1] + dp[i - 1][w - weights[i - 1]]);
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }
    return dp[n][capacity];
}
```

TRIGGER CONDITION

Asked to find the longest shared subsequence between two distinct strings or arrays (e.g., git diff calculations).

MENTAL MODEL

Think of two puzzle patterns. You align them from left to right. If characters match, your shared chain length grows. If they don't, you try sliding string A by one character, then string B by one, taking the best result.

HOW TO IMPLEMENT

1. Create a 2D array `dp[s1.length() + 1][s2.length() + 1]`.
2. Iterate through characters of both strings.
3. If `s1.charAt(i - 1) == s2.charAt(j - 1)`, set `dp[i][j] = dp[i - 1][j - 1] + 1` (grow chain).
4. If they don't match, set `dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1])` (try both slide paths).

JAVA PRODUCTION TEMPLATE

```
public int longestCommonSubsequence(String text1, String text2) {
    int m = text1.length(), n = text2.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (text1.charAt(i - 1) == text2.charAt(j - 1)) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }
    return dp[m][n];
}
```

TRIGGER CONDITION

Encountering modern Code Comprehension formats where you are handed an unfamiliar, multi-file repository with an existing bug or feature to implement.

MENTAL MODEL

Instead of writing raw algorithms in a blank text file, you are a digital detective investigating a sprawling machinery. You cannot code blindly; you must trace how wires connect (call graphs) and check data flow before picking up a wrench.

HOW TO IMPLEMENT

1. Identify the Entry Point: Locate the main class, configuration file, or system router to see where execution starts.
2. Trace the Call Graph: Follow function calls from client routes down to database adapters, noting where side-effects happen.
3. Discuss Requirements and Trade-offs: Speak out loud about how modifying Class A affects dependency Class B.

JAVA PRODUCTION TEMPLATE

```
// Trace path example in Java codebase
// 1. Identify router entry: /api/v1/cache/evict
// 2. Locate CacheController -> calls CacheManager.evictNode(key)
// 3. Trace CacheManager -> triggers LRU doubly-linked list removal
// Key IntelliJ shortcuts to discuss:
// - Cmd+B / Ctrl+B: Go to definition
// - Cmd+Alt+F7 / Alt+F7: Find usages
```

TRIGGER CONDITION

Working alongside an LLM partner in technical assessments to find edge cases or compile-time failures.

MENTAL MODEL

Think of paired programming. The AI is a junior engineer with access to all stack libraries, but lacks foresight and safety awareness. Your job is to act as the senior architect who reviews, edits, and guides their outputs.

HOW TO IMPLEMENT

1. Never let the AI code the core logic autonomously; write the skeleton and ask the AI to complete boilerplate.
2. If a bug surfaces, isolate the failing class and present the specific code block to the AI alongside the stack trace.
3. Ask targeted debugging prompts instead of generic queries.

JAVA PRODUCTION TEMPLATE

```
// Guarding against typical AI-generated ConcurrentModificationException:
// Bad AI Code: for (Node n : list) { if (n.val == 0) list.remove(n); }
// Corrected Senior Architect Code:
list.removeIf(n -> n.val == 0);
// Or use iterator explicitly:
java.util.Iterator<Node> it = list.iterator();
while (it.hasNext()) {
    if (it.next().val == 0) it.remove();
}
```

TRIGGER CONDITION

Interview rounds that allow or encourage AI assistant usage (e.g., CoderPad setups).

MENTAL MODEL

The quality of a database query depends entirely on the index. Similarly, the quality of an LLM's response depends strictly on your prompt context. A generic "find the bug" gets a generic answer; a structural constraint prompt yields an immediate fix.

HOW TO IMPLEMENT

1. Step 1: Define the persona and rules ("Act as a principal Java compiler engineer...").
2. Step 2: Provide execution context (JVM details, input bounds).
3. Step 3: Restrict outputs to specific technical criteria (e.g., zero extra heap allocations).

JAVA PRODUCTION TEMPLATE

```
// Ideal prompt construction template:
/*
Act as a principal Java compiler engineer. Audit this code block for out-of-bounds
errors or synchronization traps. Explain any identified issues and provide only
the corrected Java code within the existing class structure.
[INSERT CODE HERE]
*/
```

TRIGGER CONDITION

Need to design a bounded, fast memory cache where the least recently accessed items are evicted first.

MENTAL MODEL

Think of a small desk. You can only keep K files on it. When the desk is full and you need to review a new file, you throw out the file that you haven't opened in the longest time.

HOW TO IMPLEMENT

1. To achieve $O(1)$ get and put operations, combine a HashMap (for lookup) with a Doubly LinkedList (to maintain access order).
2. When a node is accessed (get), detach it from its current position and move it to the head.
3. When inserting (put), if cache exceeds capacity, evict the node at the tail, remove it from the HashMap, and append the new node to the head.

JAVA PRODUCTION TEMPLATE

```
class LRUCache {
    class Node {
        int key, value; Node prev, next;
        Node(int k, int v) { this.key = k; this.value = v; }
    }
    private final int capacity;
    private final java.util.Map<Integer, Node> map = new java.util.HashMap<>();
    private final Node head = new Node(0,0), tail = new Node(0,0);

    public LRUCache(int cap) {
        this.capacity = cap; head.next = tail; tail.prev = head;
    }
    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        Node node = map.get(key);
        remove(node); insertAtHead(node);
        return node.value;
    }
    public void put(int key, int val) {
        if (map.containsKey(key)) remove(map.get(key));
        Node node = new Node(key, val);
        insertAtHead(node); map.put(key, node);
        if (map.size() > capacity) {
            Node lru = tail.prev; remove(lru); map.remove(lru.key);
        }
    }
    private void remove(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }
    private void insertAtHead(Node n) {
        n.next = head.next; n.next.prev = n; head.next = n; n.prev = head;
    }
}
```

TRIGGER CONDITION

Asked to design a scalable routing mechanism to distribute database requests or web traffic across a dynamic pool of N physical servers.

MENTAL MODEL

Imagine a round roulette wheel numbered 0 to 360 degrees. Both servers and incoming request keys are hashed to a degree position on this circle. To find which server handles a request, you drop a ball on the request key's degree and walk clockwise until you hit the very first server.

HOW TO IMPLEMENT

1. Use a Java TreeMap to represent the hash ring.
2. Hash each server name (and its virtual node replicas) to an integer on the ring, and put it in the TreeMap.
3. To route a key: calculate its hash.
4. Use TreeMap.tailMap(hash) to get all servers clockwise of this position.
5. If the tailMap is empty, route to the absolute first server in the ring (wrap around).

JAVA PRODUCTION TEMPLATE

```
public class ConsistentHashRing {
    private final java.util.TreeMap<Integer, String> ring = new java.util.TreeMap<>();
    private final int replicas;

    public ConsistentHashRing(int replicas) { this.replicas = replicas; }
    public void addServer(String server) {
        for (int i = 0; i < replicas; i++) {
            ring.put((server + "-" + i).hashCode(), server);
        }
    }
    public String routeKey(String key) {
        if (ring.isEmpty()) return null;
        int hash = key.hashCode();
        if (!ring.containsKey(hash)) {
            java.util.SortedMap<Integer, String> tail = ring.tailMap(hash);
            hash = tail.isEmpty() ? ring.firstKey() : tail.firstKey();
        }
        return ring.get(hash);
    }
}
```

TRIGGER CONDITION

Need to design a system-level component that rate-limits incoming API requests to prevent server overload.

MENTAL MODEL

Think of a token dispenser. Tokens drip into a bucket at a steady rate. Every incoming API request must consume exactly 1 token to pass. If the bucket is empty, the request is instantly rejected. The maximum burst size is limited by the bucket's capacity.

HOW TO IMPLEMENT

1. Track tokens (active count) and lastRefillTimestamp.
2. On each incoming request, dynamically calculate how many tokens should have been refilled based on elapsed time: $\text{elapsedTime} * \text{refillRate}$. This avoids running a background thread!
3. If tokens are added, update lastRefillTimestamp.
4. If tokens ≥ 1 , decrement by 1 and return true (allow request); else, return false.

JAVA PRODUCTION TEMPLATE

```
public class TokenBucket {
    private final long capacity;
    private final double refillRatePerSecond;
    private double tokens;
    private long lastRefillTimestamp = System.nanoTime();

    public TokenBucket(long cap, double rate) {
        this.capacity = cap; this.refillRatePerSecond = rate; this.tokens = cap;
    }

    public synchronized boolean allowRequest() {
        long now = System.nanoTime();
        double elapsed = (now - lastRefillTimestamp) / 1e9;
        lastRefillTimestamp = now;
        tokens = Math.min(capacity, tokens + (elapsed * refillRatePerSecond));
        if (tokens >= 1.0) {
            tokens -= 1.0; return true; // Request allowed
        }
        return false; // Rate limited
    }
}
```

TRIGGER CONDITION

The absolute default strategy to execute during the 45-minute live coding round at any top-tier tech firm.

MENTAL MODEL

Writing code instantly on a whiteboard when a question is asked is a massive trap. UMPIRE ensures you communicate, scope the problem, design the algorithm, and dry-run with test cases before writing a single line.

HOW TO IMPLEMENT

1. U - Understand: Ask clarifying questions on bounds, types, size, and constraints.
2. M - Match: Name the patterns that apply (e.g., "This contiguous subarray constraint matches Sliding Window").
3. P - Plan: Write out step-by-step logic in plain English or pseudocode, and state big-O complexities.
4. I - Implement: Write syntactically clean, structured code.
5. R - Review: Walk the interviewer through your code line-by-line using a dry-run test case.
6. E - Evaluate: Discuss optimization vectors, memory trade-offs, and edge cases.

JAVA PRODUCTION TEMPLATE

```
// UMPIRE Checklist for your live whiteboard:  
// 1. Inputs: Can we have empty or negative inputs?  
// 2. Outputs: What should we return if no solution exists?  
// 3. Complexities: "I'm aiming for O(N) time and O(N) space."  
// 4. Dry Run: Walk through nums = [1, 3, 5], target = 8.  
// 5. Optimization: "We could reduce space to O(1) if we..."
```

CARD 24: STAR Method for Behavioral Loops

BEHAVIORAL STRATEG

TRIGGER CONDITION

Handled open-ended behavioral questions like "Tell me about a time you had a technical disagreement with a peer".

MENTAL MODEL

Interviewers don't want a rambling, unstructured story. STAR structures your response like a compelling movie arc: you set the stage, introduce the obstacle, describe your heroic actions, and present the quantifiable happy ending.

HOW TO IMPLEMENT

1. S - Situation: Describe the background and context in 2-3 sentences.
2. T - Task: State the specific challenge or goal you needed to accomplish.
3. A - Action: Detail what you specifically did, focusing on technical decisions, collaboration, and individual contribution.
4. R - Result: Share the outcome. Always try to provide quantifiable metrics (e.g., "reduced query latency by 45%", "saved 12 engineering hours weekly").

JAVA PRODUCTION TEMPLATE

```
// STAR Structural Outline:  
// - [S]: "At my last role, query latency on our core product dashboard  
//       exceeded 2.5 seconds under 1,000 concurrent users."  
// - [T]: "I was tasked with reducing latency to under 500ms before Q3."  
// - [A]: "I traced the DB call graph, implemented a Redis cache with an  
//       LRU eviction policy, and added a compound database index."  
// - [R]: "Latency dropped to 210ms (91% reduction) and dashboard  
//       concurrency grew by 3x without scaling server footprint."
```

CARD 25: High-Leverage Salary Negotiation

CAREER STRATEG

TRIGGER CONDITION

Once you receive an offer from a top-tier tech firm and need to maximize your compensation package.

MENTAL MODEL

Recruiters represent the company's budget, not your financial interests. Your leverage does not come from complaining about cost of living, but from competing market alternatives, specialized skill value, and professional scripting.

HOW TO IMPLEMENT

1. Never Share Your Number First: Let the recruiter make the initial offer. Sharing a number first caps your potential upside.
2. Leverage Alternates: Use active conversations or offers at parallel firms as a baseline for competition.
3. Focus on Total Compensation: Negotiate across base salary, sign-on bonuses, and stock grants (RSUs).

JAVA PRODUCTION TEMPLATE

```
// Negotiation Scripting Template:  
/*  
"Thank you for the offer, I am incredibly excited about the team.  
However, I am currently in final rounds with Meta and have a strong  
competing offer from Amazon. If we can adjust the equity target  
to $Y, I am ready to sign the offer today."  
*/
```